Homework #3

Due: March 23, 2026 at 11:59 PM

Neural Networks and Kernels

Introduction

This homework is about neural networks and kernels.

1. You'll explore how kernel functions implicitly define feature maps and how kernel ridge regression connects to predictions as weighted sums of training labels.
2. You'll derive the backpropagation algorithm for a single-hidden-layer neural network for the binary classification task.
3. You'll discover neural scaling laws empirically and derive optimal compute allocation strategies.

As always, please start early and come to office hours with questions!

Resources and Submission Instructions

You may want to consider the notes from Section 4 (Richer Features and Neural Networks).

Please type your solutions after the corresponding problems using this LaTeX template, and start each problem on a new page.

Please submit the writeup PDF to the Gradescope assignment 'HW3'. Remember to assign pages for each question. **You must include any plots in your writeup PDF.** . Please submit your LaTeXfile and code files to the Gradescope assignment 'HW3 - Supplemental.' The supplemental files will only be checked in special cases, e.g. honor code issues, etc. Your files should be named in the same way as we provide them in the repository, e.g. `hw3.pdf`, etc.

Problem 1 (Kernels and Feature Maps, 30 pts)

In this problem, we explore how kernel functions implicitly define feature maps and how this leads to predictions that are weighted sums of training labels.

Recall from lecture that a kernel function $K(\mathbf{x}, \mathbf{x}')$ computes an inner product in some feature space: $K(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^\top \phi(\mathbf{x}')$, where ϕ is a feature map.

1. **Expanding the polynomial kernel.** Consider the degree-2 polynomial kernel for $\mathbf{x} \in \mathbb{R}^2$:

$$K(\mathbf{x}, \mathbf{x}') = (1 + \mathbf{x}^\top \mathbf{x}')^2.$$

   (a) Writing $\mathbf{x} = (x_1, x_2)$ and $\mathbf{x}' = (x_1', x_2')$, expand $(1 + x_1 x_1' + x_2 x_2')^2$ and collect terms. Show that the result can be written as $\phi(\mathbf{x})^\top \phi(\mathbf{x}')$ for some feature map $\phi : \mathbb{R}^2 \to \mathbb{R}^6$. State ϕ explicitly.

   (b) Verify your answer with a concrete example: compute both $K(\mathbf{x}, \mathbf{x}')$ and $\phi(\mathbf{x})^\top \phi(\mathbf{x}')$ for $\mathbf{x} = (1, 0)$ and $\mathbf{x}' = (1, 1)$.

2. **Visualizing the feature map.** We provide a synthetic dataset of concentric circles: an inner ring (class 0) and an outer ring (class 1). See the accompanying notebook for the data generation code.

   (a) Plot the dataset in the original space $\mathbb{R}^2$. Are the data linearly separable? Explain briefly.

   (b) Apply the feature map ϕ from Part 1 to the data. Produce a 3D scatter plot of $(x_1,\ x_2,\ x_1^2 + x_2^2)$ with points colored by class. Are the data linearly separable in feature space? Why does this feature map help for concentric circles?

   (c) Fit a ridge regression model in the feature space:

$$\mathbf{w} = (\mathbf{\Phi}^\top \mathbf{\Phi} + \lambda \mathbf{I})^{-1} \mathbf{\Phi}^\top \mathbf{y},$$

   where $\mathbf{\Phi} \in \mathbb{R}^{N \times 6}$ is the feature matrix with $\phi(\mathbf{x}_n)^\top$ as its nth row and $\mathbf{y} \in \{0, 1\}^N$ are the labels. Use $\lambda = 0.01$. Use pure `numpy` (no `sklearn`). Plot the decision boundary over the training data.

3. **From feature space to kernel space.**

   In Part 2c you computed $\mathbf{w}$ and made predictions as $f(\mathbf{x}^*) = \phi(\mathbf{x}^*)^\top \mathbf{w}$, which required explicitly constructing the feature matrix $\mathbf{\Phi}$. We now show that the same predictions can be written as a weighted sum over training points, purely in terms of the kernel function K.

   (a) Let $\mathbf{K} \in \mathbb{R}^{N \times N}$ be the kernel matrix with entries $K_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$, and define the weight vector $\alpha = (\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{y}$. Show that the ridge regression prediction at a new point $\mathbf{x}^*$ can be written as:

$$f(\mathbf{x}^*) = \sum_{n=1}^{N} \alpha_n \, K(\mathbf{x}_n, \mathbf{x}^*).$$

   Hint: Define $\mathbf{w} = \mathbf{\Phi}^\top \alpha$ and verify that this $\mathbf{w}$ satisfies the ridge regression normal equations $(\mathbf{\Phi}^\top \mathbf{\Phi} + \lambda \mathbf{I})\mathbf{w} = \mathbf{\Phi}^\top \mathbf{y}$, using $\mathbf{\Phi}\mathbf{\Phi}^\top = \mathbf{K}$.

   (b) Implement kernel ridge regression using the polynomial kernel $K(\mathbf{x}, \mathbf{x}') = (1 + \mathbf{x}^\top \mathbf{x}')^2$. Compute $\alpha = (\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{y}$ on the training data. Then make a scatter plot of the training data where each point is sized proportionally to $|\alpha_n|$.

   (c) Which points have the largest $|\alpha_n|$? Where do they tend to be located? Briefly explain why.

(d) In kernel ridge regression, every training point contributes to the prediction (all $\alpha_n \neq 0$). In a *support vector machine* (SVM), only the training points closest to the decision boundary have $\alpha_n \neq 0$. These points are called *support vectors*.

Looking at your plot from part (b), which training points do you think would be selected as support vectors? Why might it be useful to have most $\alpha_n = 0$?

4. **The RBF kernel.**

The radial basis function (RBF) kernel is defined as:

$$K(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\sigma^2}\right).$$

Unlike the polynomial kernel, whose degree limits it to polynomial decision boundaries, the RBF kernel can approximate any continuous decision boundary. The bandwidth σ controls how quickly $K(\mathbf{x}, \mathbf{x}')$ decays with the distance between $\mathbf{x}$ and $\mathbf{x}'$.

(a) Expand $\|\mathbf{x} - \mathbf{x}'\|^2$ and use the result to write $K(\mathbf{x}, \mathbf{x}')$ as a product of three exponentials: one depending only on $\mathbf{x}$, one only on $\mathbf{x}'$ and one involving $\mathbf{x}^\mathsf{T}\mathbf{x}'$.

(b) The other two factors depend on $\mathbf{x}$ or $\mathbf{x}'$ alone, so they act as scalar weights on each input. The factor involving $\mathbf{x}^\mathsf{T}\mathbf{x}'$ is what couples the two inputs. Expand it as a Taylor series.

(c) The kernel trick lets us work with K without ever computing ϕ explicitly. Why is this essential for the RBF?

(d) Assume the training points are distinct. With $\alpha = (\mathbf{K}+\lambda\mathbf{I})^{-1}\mathbf{y}$ and $f(\mathbf{x}^*) = \sum_{n=1}^{N} \alpha_n K(\mathbf{x}_n, \mathbf{x}^*)$, show that as $\sigma \to 0$ the model classifies every training point correctly but predicts $f(\mathbf{x}^*) \to 0$ at any point not in the training set.

(e) Show that as $\sigma \to \infty$ the model predicts the same value at every point.

Solution:

1.

(a) Expanding $(1 + x_1x_1' + x_2x_2')^2$ gives us:

$$\begin{aligned}(1 + x_1x_1' + x_2x_2')^2 &= 1^2 + (x_1x_1')^2 + (x_2x_2')^2 + 2(1)(x_1x_1') + 2(1)(x_2x_2') + 2(x_1x_1')(x_2x_2') \\ &= 1 + 2x_1x_1' + 2x_2x_2' + x_1^2(x_1')^2 + x_2^2(x_2')^2 + 2x_1x_2x_1'x_2'\end{aligned}$$

Now we can define the feature map $\phi : \mathbb{R}^2 \to \mathbb{R}^6$ by

$$\phi(x) = \big(1,\ \sqrt{2}\,x_1,\ \sqrt{2}\,x_2,\ x_1^2,\ x_2^2,\ \sqrt{2}\,x_1x_2\big)^\top$$

This gives us:

$$\phi(x)^\top\phi(x') = 1 \cdot 1 + 2x_1x_1' + 2x_2x_2' + x_1^2(x_1')^2 + x_2^2(x_2')^2 + 2x_1x_2x_1'x_2' = K(x, x')$$

(b) For $x = (1, 0)$ and $x' = (1, 1)$:

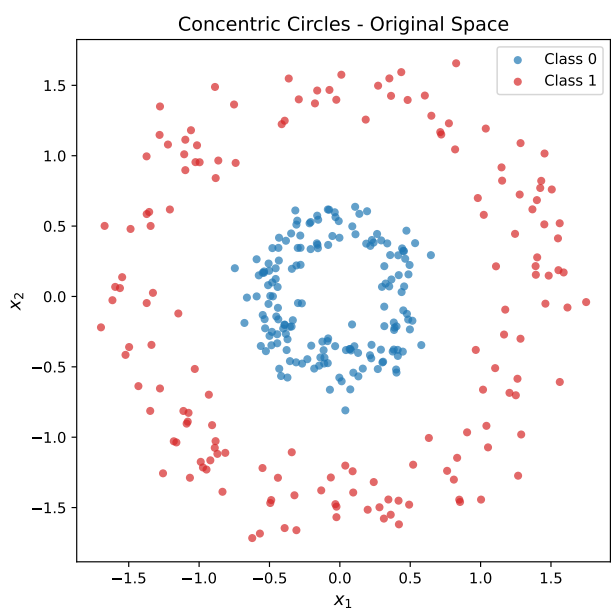
$$K(x, x') = (1 + 1 \cdot 1 + 0 \cdot 1)^2 = (1 + 1)^2 = 4$$

$\phi(x) = (1, \sqrt{2}, 0, 1, 0, 0)^\top$ and $\phi(x') = (1, \sqrt{2}, \sqrt{2}, 1, 1, \sqrt{2})^\top$, so

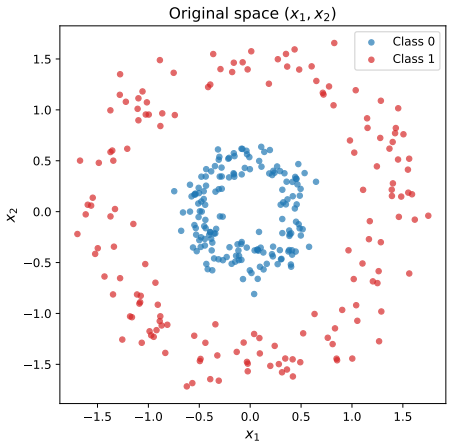
$$\phi(x)^\top\phi(x') = 1 + 2 + 0 + 1 + 0 + 0 = 4$$

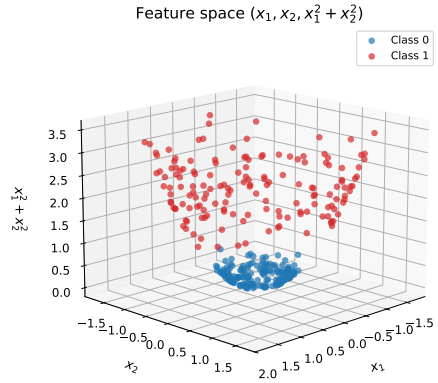
2.

(a) The data are not linearly separable in $\mathbb{R}^2$. The inner ring (class 0) and outer ring (class 1) are concentric circles, so any linear boundary would cut through both rings and misclassify points. We can see this in the plot:

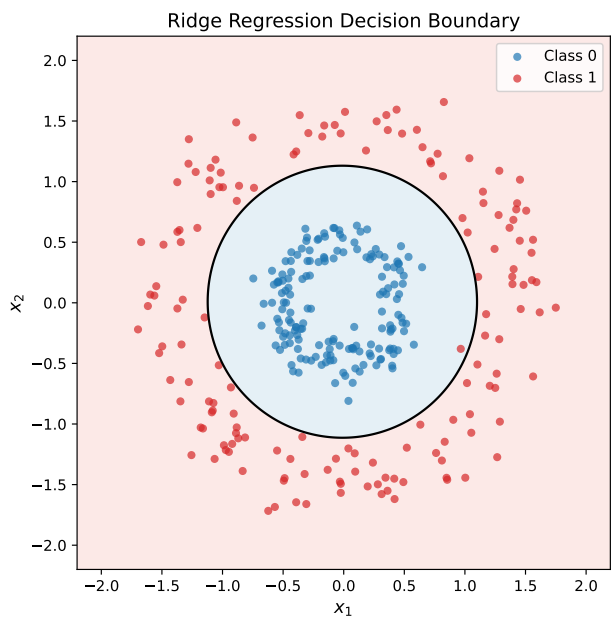


(b) In feature space, the third coordinate is $x_1^2 + x_2^2$ which is squared radius. Class 0 has smaller r^2 and class 1 has larger r^2, so a horizontal plane can separate them. The polynomial feature map helps because concentric circles differ by radius and the mapping makes radius explicit, turning a nonlinear boundary in 2D into a linear one in 3D. We can see this in the plot:





(c) The decision boundary for the specified ridge regression model is:



3.

(a) We can define $w = \Phi^\top \alpha$. The ridge normal equations are $(\Phi^\top \Phi + \lambda I)w = \Phi^\top y$. By substituting $w = \Phi^\top \alpha$:

$$(\Phi^\top \Phi + \lambda I)\Phi^\top \alpha = \Phi^\top y$$

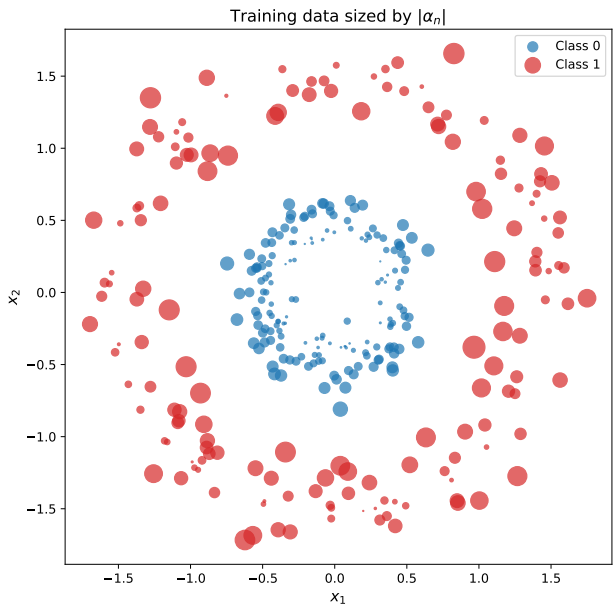
Left-multiply by Φg gives us $\Phi(\Phi^\top \Phi + \lambda I)\Phi^\top \alpha = \Phi\Phi^\top y$, and using $\Phi\Phi^\top = K$:

$$(K + \lambda I)\alpha = y \Rightarrow \alpha = (K + \lambda I)^{-1} y$$

The prediction is $f(x^*) = \phi(x^*)^\top w = \phi(x^*)^\top \Phi^\top \alpha$. The nth entry of $\Phi\phi(x^*)$ is $\phi(x_n)^\top \phi(x^*) = K(x_n, x^*)$, so:

$$f(x^*) = \sum_{n=1}^{N} \alpha_n \, K(x_n, x^*)$$

(b) The training points sized by $|\alpha_n|$:



(c) Points with the largest $|\alpha_n|$ tend to be near the decision boundary (the region between the inner and outer rings) or in regions where the two classes overlap. These points are most influential because they lie where the model is uncertain, small changes in their labels would most affect the fit.

(d) Support vectors would be the points near the boundary, which are the largest $|\alpha_n|$ in our plot. Having most $\alpha_n = 0$ is useful for sparsity. Predictions depend only on a few points, reducing storage and computation at test time.

4.

(a) We can expand $\|x - x'\|^2 = \|x\|^2 + \|x'\|^2 - 2x^\top x'$ as follows:

$$K(x, x') = \exp\left(-\frac{\|x\|^2}{2\sigma^2}\right) \exp\left(-\frac{\|x'\|^2}{2\sigma^2}\right) \exp\left(\frac{x^\top x'}{\sigma^2}\right)$$

(b) The factor $\exp(x^\top x'/\sigma^2)$ has Taylor expansion:

$$\exp\left(\frac{x^\top x'}{\sigma^2}\right) = \sum_{k=0}^{\infty} \frac{1}{k!} \left(\frac{x^\top x'}{\sigma^2}\right)^k$$

(c) The RBF kernel corresponds to an infinite dimensional feature map (the Taylor series has infinitely many terms). Computing ϕ explicitly is impossible. The kernel trick lets us compute $K(x, x')$ in $O(d)$ time without ever constructing ϕ.

(d) As $\sigma \to 0$, $K(x_i, x_j) \to \delta_{ij}$ (1 if $i = j$, 0 otherwise) for distinct training points. So $K \to I$ and $\alpha \to (I + \lambda I)^{-1} y = y/(1 + \lambda)$. For training point x_n, $f(x_n) = \sum_m \alpha_m K(x_m, x_n) \to \alpha_n \cdot 1 = y_n/(1 + \lambda)$, so correct classification (threshold 0.5). For $x^* \notin \{x_1, \ldots, x_N\}$, $K(x_n, x^*) \to 0$ for all n, so $f(x^*) \to 0$.

(e) As $\sigma \to \infty$, $\|x - x'\|^2/(2\sigma^2) \to 0$, so $K(x, x') \to 1$ for all x, x'. Thus $K \to \mathbf{1}\mathbf{1}^\top$ (all ones) and $f(x^*) = \sum_n \alpha_n \cdot 1 = \mathbf{1}^\top \alpha$ is the same for every x^*.

Problem 2 (Neural Networks, 20 pts)

In this problem, we will take a closer look at how gradients are calculated for backpropagation with a simple multi-layer perceptron (MLP). The MLP will consist of a first fully connected layer with a sigmoid activation, followed by a one-dimensional, second fully connected layer with a sigmoid activation to get a prediction for a binary classification problem. We use non-linear activation functions as the composition of linear functions is linear. Assume bias has not been merged. Let:

- $\mathbf{W}_1$ be the weights of the first layer, $\mathbf{b}_1$ be the bias of the first layer.
- $\mathbf{W}_2$ be the weights of the second layer, $\mathbf{b}_2$ be the bias of the second layer.

The described architecture can be written mathematically as:

$$\hat{y} = \sigma(\mathbf{W}_2 \left[\sigma\left(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1\right)\right] + \mathbf{b}_2)$$

where $\hat{y}$ is a scalar output of the net when passing in the single datapoint $\mathbf{x}$ (represented as a column vector), the additions are element wise additions, and the sigmoid is an element wise sigmoid.

1. Let:

   - N be the number of datapoints we have
   - M be the dimensionality of the data
   - H be the size of the hidden dimension of the first layer. Here, hidden dimension is used to describe the dimension of the resulting value after going through the layer. Based on the problem description, the hidden dimension of the second layer should be 1.

   Write out the dimensionality of each of the parameters, and of the intermediate variables:

$$\mathbf{a}_1 = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1, \qquad \mathbf{z}_1 = \sigma(\mathbf{a}_1)$$
$$a_2 = \mathbf{W}_2\mathbf{z}_1 + \mathbf{b}_2, \qquad \hat{y} = z_2 = \sigma(a_2)$$

   and make sure they work with the mathematical operations described above. Examining shapes is one of the key ways to debug your code, and can be done using .shape after any numpy array.

2. We will derive the gradients for each of the parameters, which can then be used along with gradient descent to find weights that improve our model's performance. For this question, assume there is only one datapoint $\mathbf{x}$, and that our loss is $L = -(y\log(\hat{y}) + (1-y)\log(1-\hat{y}))$. For all questions, the chain rule will be useful.

   (a) Find $\frac{\partial L}{\partial b_2}$.

   (b) Find $\frac{\partial L}{\partial W_2^h}$, where W_2^h represents the hth element of $\mathbf{W}_2$.

   (c) Find $\frac{\partial L}{\partial b_1^h}$, where b_1^h represents the hth element of $\mathbf{b}_1$. (**Hint**: Note that only the hth element of $\mathbf{a}_1$ and $\mathbf{z}_1$ depend on b_1^h - this should help you with how to use the chain rule.)

   (d) Find $\frac{\partial L}{\partial W_1^{h,j}}$, where $W_1^{h,j}$ represents the element in row h, column j in $\mathbf{W}_1$.

Problem 2 (cont.)

3. We now explore an example of forward-mode auto-differentiation. Consider the following equation:

$$f(x_1, x_2) = \ln(\sin(x_1)) + x_1 \exp\{x_2\}$$

This equation can be split up using intermediate variables $v_1, \ldots, v_7$ as follows:

$$\begin{aligned} v_1 &= x_1 \\ v_2 &= \sin(v_1) \\ v_3 &= \ln(v_2) \\ v_4 &= x_2 \\ v_5 &= \exp\{v_4\} \\ v_6 &= v_1 v_5 \\ v_7 &= v_3 + v_6 \\ f(x_1, x_2) &= v_7 \end{aligned}$$

Splitting up the equation like this is very similar to what an auto-differentiation library would do. From these equations we can construct a *computational graph* where each node of the graph corresponds to an input, an intermediate variable, or the output.

(a) Let $x_1 = \frac{\pi}{6}$ and $x_2 = 1$. Calculate the values of all the intermediate variables $v_1, \ldots v_7$ and $f(x_1, x_2)$.

(b) Calculate the derivative of all of the intermediate variables $v_1, \ldots, v_7$ and f with respect to x_1 evaluated at $x_1 = \frac{\pi}{6}$ and $x_2 = 1$. Remember to write out the equations before evaluating, e.g.,

$$\frac{\partial f(x)}{\partial x} = \frac{\partial f(x)}{\partial g(x)} \frac{\partial g(x)}{\partial x}.$$

Solution: **1.**

We can think of $\mathbf{x}$ as a column vector in $\mathbb{R}^M$. If we want every matrix multiply to line up, we need:

- $W_1 \in \mathbb{R}^{H\times M}$, $b_1 \in \mathbb{R}^H$
- $a_1 = W_1 x + b_1 \in \mathbb{R}^H$, $z_1 = \sigma(a_1) \in \mathbb{R}^H$
- $W_2 \in \mathbb{R}^{1\times H}$, $b_2 \in \mathbb{R}$
- $a_2 = W_2 z_1 + b_2 \in \mathbb{R}$, $\hat{y} = \sigma(a_2) \in \mathbb{R}$

2.

We use binary cross-entropy: $L = -(y \log \hat{y} + (1-y)\log(1-\hat{y}))$.

(a) If we follow the chain from L to $\hat{y}$ to a_2, first $\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} = \frac{\hat{y}-y}{\hat{y}(1-\hat{y})}$, and next $\frac{\partial \hat{y}}{\partial a_2} = \sigma'(a_2) = \hat{y}(1-\hat{y})$. Because $a_2 = W_2 z_1 + b_2$, we have $\frac{\partial a_2}{\partial b_2} = 1$. Multiplying the pieces gives us:

$$\frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial a_2}\frac{\partial a_2}{\partial b_2} = \frac{\hat{y}-y}{\hat{y}(1-\hat{y})}\cdot \hat{y}(1-\hat{y})\cdot 1 = \hat{y}-y$$

(b) If we write out $a_2 = \sum_{h'} W_2^{h'} z_1^{h'} + b_2$, the derivative with respect to one weight is simply $\frac{\partial a_2}{\partial W_2^h} = z_1^h$. Chaining as before, we get:

$$\frac{\partial L}{\partial W_2^h} = \frac{\partial L}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial a_2}\frac{\partial a_2}{\partial W_2^h} = (\hat{y}-y)\cdot z_1^h$$

(c) Only a_1^h and z_1^h depend on b_1^h. If we walk the graph $L \to \hat{y} \to a_2 \to z_1^h \to a_1^h \to b_1^h$, we use $\frac{\partial L}{\partial a_2} = \hat{y}-y$, $\frac{\partial a_2}{\partial z_1^h} = W_2^h$, $\frac{\partial z_1^h}{\partial a_1^h} = \sigma'(a_1^h) = z_1^h(1-z_1^h)$, and $\frac{\partial a_1^h}{\partial b_1^h} = 1$. Putting it together, we get:

$$\frac{\partial L}{\partial b_1^h} = (\hat{y}-y)\cdot W_2^h \cdot z_1^h(1-z_1^h)$$

(d) Since $a_1^h = \sum_{j'} W_1^{h,j'} x_{j'} + b_1^h$, we get $\frac{\partial a_1^h}{\partial W_1^{h,j}} = x_j$. We can reuse the gradient from part (c):

$$\frac{\partial L}{\partial W_1^{h,j}} = \frac{\partial L}{\partial b_1^h}\cdot\frac{\partial a_1^h}{\partial W_1^{h,j}} = (\hat{y}-y)\cdot W_2^h \cdot z_1^h(1-z_1^h)\cdot x_j$$

3.

(a) If we plug in $x_1 = \frac{\pi}{6}$ and $x_2 = 1$, we can just evaluate the graph step by step:

$$\begin{aligned}
&v_1 = x_1 = \frac{\pi}{6}, \quad v_2 = \sin(v_1) = \frac{1}{2}, \quad v_3 = \ln(v_2) = \ln(1/2) = -\ln 2,\\
&v_4 = x_2 = 1, \quad v_5 = e^{v_4} = e, \quad v_6 = v_1 v_5 = \frac{\pi e}{6}, \quad v_7 = v_3 + v_6 = -\ln 2 + \frac{\pi e}{6},\\
&f = v_7 = -\ln 2 + \frac{\pi e}{6} \approx -0.693 + 1.423 = 0.730
\end{aligned}$$

(b) If we want $\partial/\partial x_1$, we propagate sensitivities forward through the same graph:

$$\begin{aligned}
&\frac{\partial v_1}{\partial x_1} = 1, \quad \frac{\partial v_2}{\partial x_1} = \cos(v_1)\frac{\partial v_1}{\partial x_1} = \cos(\pi/6) = \frac{\sqrt{3}}{2},\\
&\frac{\partial v_3}{\partial x_1} = \frac{1}{v_2}\frac{\partial v_2}{\partial x_1} = \frac{2\cdot\sqrt{3}/2}{1} = \sqrt{3},\\
&\frac{\partial v_4}{\partial x_1} = 0, \quad \frac{\partial v_5}{\partial x_1} = 0, \quad \frac{\partial v_6}{\partial x_1} = v_5\frac{\partial v_1}{\partial x_1} + v_1\frac{\partial v_5}{\partial x_1} = e\cdot 1 = e,\\
&\frac{\partial v_7}{\partial x_1} = \frac{\partial v_3}{\partial x_1} + \frac{\partial v_6}{\partial x_1} = \sqrt{3}+e, \quad \frac{\partial f}{\partial x_1} = \sqrt{3}+e \approx 1.732 + 2.718 = 4.450
\end{aligned}$$

Problem 3 (Neural Scaling Laws, 50 pts)

In this problem, we look pr an observation that is extremely relevant and widely utilized in industry and research.

Large-scale experiments have revealed a striking observation: the test loss of neural networks follows **power laws** in model size and dataset size. In this problem, you will discover these laws yourself on a smaller scale, use them to make predictions, and derive the optimal way to allocate a fixed compute budget. Empirically, for many architectures and datasets, test loss L follows:

$$L(N, D) \approx \frac{a}{N^\alpha} + \frac{b}{D^\beta} + L_\infty \tag{1}$$

where N is the number of trainable parameters, D is the number of training samples, and L_∞ is the irreducible error (the best loss any model could achieve on this task). On a log-log plot, each term appears as a straight line. There is no first-principles derivation of *why* this particular functional form holds, it is an empirical observation. However, the form makes intuitive sense through the lens of bias-variance:

- a/N^α: **approximation error**. A small model cannot represent the true function well. As we add parameters, this error decreases — but with diminishing returns (power law, not exponential).
- b/D^β: **estimation error**. Even a large model cannot learn well from too little data. More data reduces this error, again with diminishing returns.
- L_∞: **irreducible error**. Some loss remains no matter how large the model or dataset — due to label ambiguity, insufficient input information (e.g., low-resolution images), or inherent noise.

Setup: Architecture

You will use a depth-scaled residual CNN (ResNet) with a fixed base channel width $C = 64$ and a variable number of residual blocks K:

Stem: $\texttt{Conv2d}(1, C, 3, \text{padding} = 1, \text{bias} = \text{False}) \to \texttt{BatchNorm2d}(C) \to \texttt{ReLU} \to \texttt{MaxPool2d}(2)$

$K\times$ **ResBlock:** each block consists of

$$\texttt{Conv2d}(C, C, 3, \text{padding} = 1, \text{bias} = \text{False}) \to \texttt{BatchNorm2d}(C) \to \texttt{ReLU}$$
$$\to \texttt{Conv2d}(C, C, 3, \text{padding} = 1, \text{bias} = \text{False}) \to \texttt{BatchNorm2d}(C) + \text{skip} \to \texttt{ReLU}$$

Head: $\texttt{AdaptiveAvgPool2d}(1) \to \texttt{Flatten} \to \texttt{Linear}(C, 10)$

Fashion-MNIST images are 28×28 with 1 channel. After the stem's max-pool the spatial size is 14×14; the residual blocks preserve this size; the adaptive average pool reduces it to 1×1. You are free to use `torch.nn` and `torchvision` for this problem.

(a) **[22 pts]** Train ResNets on Fashion-MNIST at varying scales.

(i) **[10 pts]** Write out the total parameter count N as a function of K (number of residual blocks) for the architecture above. Be careful: each `Conv2d` with `bias=False` has $\text{kernel}^2 \times C_{\text{in}} \times C_{\text{out}}$ parameters, each $\texttt{BatchNorm2d}(C)$ has $2C$ parameters (scale and shift), and the linear layer has in_features $\times$ out_features + out_features. Write the training code for the ResNet. The values for the different K values are provided for your convenience.

(ii) **[6 pts]** Fix $K = 12$ and train on random subsets of size $D \in \{1000, 2000, 5000, 10000, 20000, 30000, 50000\}$. Record the final test loss for each.

(iii) **[6 pts]** Make two log-log plots:

- $\log(L - \hat{L}_\infty)$ vs. $\log N$ (at fixed D)
- $\log(L - \hat{L}_\infty)$ vs. $\log D$ (at fixed N)

where $\hat{L}_\infty$ is estimated as the loss of the best configuration *within each sweep* (i.e., the $K = 18$ run for the model-size plot, and the $D = 50000$ run for the data-size plot). This anchor point is excluded from the fit in each case. Fit a line to each plot. Report your estimated exponents $\hat{\alpha}$ and $\hat{\beta}$, the R^2 values, as well as your plots, down below. Discuss briefly: why do we need to subtract $\hat{L}_\infty$ before taking the log? What would the plot look like if we skipped this step? Feel free to modify the graphing code to see what it would look like, though that is not necessary.

(b) **[10 pts]** Suppose training costs C_{compute} FLOPs, and assume the approximation $C_{\text{compute}} \approx 6ND$ (forward + backward pass $\approx$ 6 multiply-adds per parameter per sample).

(i) **[5 pts]** For a fixed compute budget C_{compute}, we want to choose N and D to minimize the loss in Eq. (1). Using the constraint $D = C_{\text{compute}}/(6N)$, substitute into the loss and minimize over N. Show that the compute-optimal parameter count satisfies:

$$N^*(C_{\text{compute}}) \propto C_{\text{compute}}^{\beta/(\alpha+\beta)}$$

and the compute-optimal dataset size satisfies:

$$D^*(C_{\text{compute}}) \propto C_{\text{compute}}^{\alpha/(\alpha+\beta)}.$$

(ii) **[3 pts]** Plug in your empirically estimated $\hat{\alpha}$ and $\hat{\beta}$ from part (a). If you double your compute budget, by what factor should you increase N? By what factor should you increase D?

(iii) **[2 pts]** The "Chinchilla" result (Hoffmann et al., 2022) found $\alpha \approx \beta$ for large language models, implying N and D should be scaled equally. If instead $\alpha \gg \beta$ (model size matters much more than data), what would the optimal strategy look like? What about $\alpha \ll \beta$?

(c) **[8 pts]** This is the key test of whether scaling laws are actually useful: can you predict the performance of a model you haven't trained yet?

(i) **[4 pts]** Using *only* your fitted scaling law from part (a), predict the test loss at $D = 60000$. Then actually train that model and report the true test loss. How close was your prediction? Discuss potential reasons for any discrepancy.
Hint: What problems may arise from using $D = 50000$ as your irreducible loss, and what value does it get?

(ii) **[4 pts]** The $K = 18$ model trained on the full dataset achieved a test cross-entropy of 0.2195. Suppose you are limited to the $K = 12$ architecture. According to your fitted data scaling law, how much training data would $K = 12$ need to match $K = 18$'s performance? Is this achievable? What does this tell you about the relationship between model capacity and data?

Solution: (a)(i) If we add up parameters block by block, it is just careful bookkeeping:

Stem: Conv2d$(1, C, 3)$: $3^2 \cdot 1 \cdot C = 9C$, BatchNorm2d(C): $2C$, so the stem contributes $11C$.

Each ResBlock: Two Conv2d$(C, C, 3)$: $2{\cdot}9{\cdot}C^2 = 18C^2$, Two BatchNorm2d$(C)$: $4C$, per block: $18C^2{+}4C$.

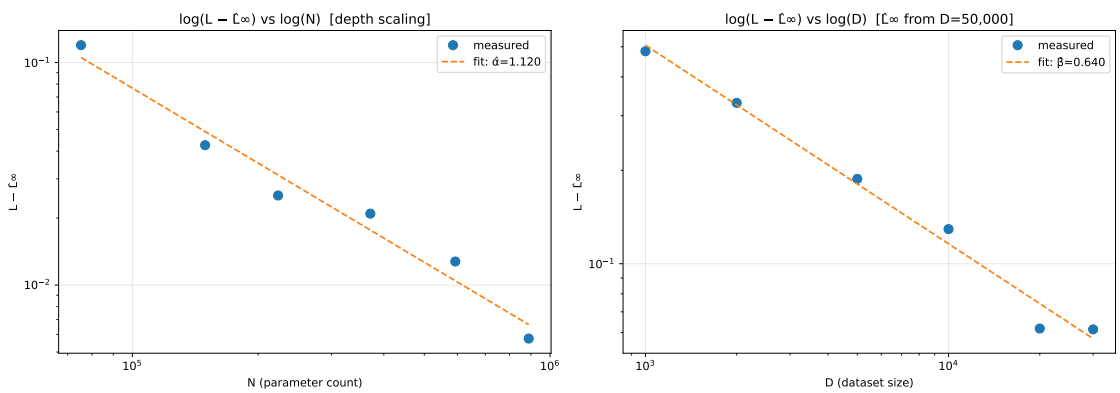
Head: Linear$(C, 10)$: $C \cdot 10 + 10 = 10C + 10$, so the head contributes $10C + 10$.

Putting it together, $N(K) = 11C + K(18C^2 + 4C) + 10C + 10 = 21C + 10 + K(18C^2 + 4C)$. If we plug in $C = 64$, we get $N(K) = 1354 + K \cdot 73984$.

(a)(ii) With $K = 12$, the results are as follows:

D	test CE
1000	0.71495
2000	0.56106
5000	0.41887
10000	0.36048
20000	0.29303
30000	0.29265
50000	0.23118

(a)(iii) Here are the log-log plots and fits from the experiments:



From the linear fits on $\log(L - \hat{L}_\infty)$ (with $\hat{L}_\infty$ taken from the $K = 18$ run for the model size sweep and from the $D = 50000$ run for the data size sweep, and those anchors excluded from each fit), we obtain $\hat{\alpha} = 1.120$ with $R^2 = 0.968$ for the model size curve, and $\hat{\beta} = 0.640$ with $R^2 = 0.985$ for the data size curve.

We need to subtract $\hat{L}_\infty$ because if we plot $\log L$ against $\log N$ or $\log D$, the curve bends as L gets close to L_∞, so we do not see a clean straight line on log-log paper. If we subtract $\hat{L}_\infty$ first, then $\log(L - \hat{L}_\infty) \approx \log(a) - \alpha \log N$, which is a line with slope $-\alpha$. If we skip the subtraction, the plot tends to sag downward instead of following a power law.

(b)(i) Suppose we fix compute and use $D = C_{\text{compute}}/(6N)$. If we substitute that into Eq(1), the loss becomes a function of N alone:

$$L(N) = \frac{a}{N^\alpha} + \frac{b}{(C/(6N))^\beta} + L_\infty = \frac{a}{N^\alpha} + \frac{b \cdot 6^\beta}{C^\beta} N^\beta + L_\infty$$

If we take a derivative with respect to N and set it to zero, we get

$$-\frac{a\alpha}{N^{\alpha+1}} + \frac{b \cdot 6^\beta}{C^\beta} \beta N^{\beta-1} = 0 \Rightarrow \frac{a\alpha}{N^{\alpha+1}} = \frac{b \cdot 6^\beta \beta}{C^\beta} N^{\beta-1}$$

Rearranging shows how N must scale with the budget:

$$N^{\alpha+\beta} \propto C^\beta \Rightarrow N^* \propto C^{\beta/(\alpha+\beta)}, D^* = \frac{C}{6N^*} \propto C^{\alpha/(\alpha+\beta)}$$

(b)(ii) If we double the compute budget C_{compute}, then N^* scales by $2^{\beta/(\alpha+\beta)}$ and D^* by $2^{\alpha/(\alpha+\beta)}$. Plugging in $\hat{\alpha} \approx 1.12$ and $\hat{\beta} \approx 0.64$ gives $\hat{\alpha}+\hat{\beta} \approx 1.76$, so N^* should multiply by about $2^{0.64/1.76} \approx 2^{0.36} \approx 1.29$ and D^* by about $2^{1.12/1.76} \approx 2^{0.64} \approx 1.55$. Here the model size exponent is larger than the data exponent, so optimal scaling allocates more compute to dataset size than to parameters when the budget doubles.

(b)(iii) If $\alpha \gg \beta$, then $\beta/(\alpha+\beta)$ is small and $\alpha/(\alpha+\beta)$ is close to 1, so N^* grows only slowly with compute while D^* grows almost proportionally, the compute-optimal strategy is to scale the dataset faster than the parameter count (more FLOPs on data than on model). Intuitively, a large α means approximation error drops quickly as N increases, while a small β means each extra training example reduces estimation error slowly, so you need many samples. If $\alpha \ll \beta$, the exponents reverse, N^* should track the budget more closely and D^* grows slowly, so you prioritize larger models over more data.

(c)(i) Using only the data scaling fit from part (a), the predicted test loss at $D = 60000$ (for $K = 12$) is $\hat{L}(60000) = \hat{L}_\infty^{(D)} + \hat{B} \cdot 60000^{-\hat{\beta}} \approx 0.268$. Training $K = 12$ on the full $D = 60000$ set gave a true best test cross entropy of about 0.221, so the prediction was off by roughly 0.047 in absolute terms. The large gap is expected as we extrapolated beyond the largest D used in the fit, and using the $D = 50000$ run as $\hat{L}_\infty^{(D)}$ can overestimate the floor (that run still carries substantial finite data error). The actual run at $D = 60000$ also shows that the simple two term fit misses the bend toward the true irreducible loss.

(c)(ii) To match the $K = 18$ run's test loss ≈ 0.2195, we would ask for D such that $\hat{L}_\infty^{(D)} + \hat{B}D^{-\hat{\beta}} = 0.2195$ under the same data only fit. Here $\hat{L}_\infty^{(D)} \approx 0.231$ from the best $K = 12$ run at $D = 50000$ already exceeds 0.2195, so the fitted equation has no positive solution for D. Algebraically, the model predicts that data scaling alone cannot push $K = 12$ below that target. In practice, training $K = 12$ on all $D = 60000$ images achieved test CE ≈ 0.221, nearly the $K = 18$ number, so the fitted $\hat{L}_\infty^{(D)}$ was too pessimistic. Fashion-MNIST caps at 60,000 training points anyway, so we could not test an even larger D. The lesson is that when approximation error dominates (small K), a larger model can reach a lower loss than a smaller one at the same D, data scaling can close much of the gap, but the pure power law fit on $D \leq 50000$ does not capture the joint (N, D) behavior perfectly.

Name: Mert Iravul
Collaborators and Resources: